

Занятие 1. С чего начать

Для кого этот документ. Вы умеете программировать, но агентами в работе ещё не пользовались — или пробовали пару раз чат в браузере. На занятии всё показывали в терминале, и со стороны это могло выглядеть как обязательное условие. Это не так, и первый же раздел про это.

Здесь нет полноты — здесь порядок действий. Прочитать можно за полчаса, сделать по нему — за вечер.

Комплект материалов занятия

ДОКУМЕНТ	КОГДА ОТКРЫВАТЬ
<code>getting-started</code> — этот файл	Сейчас. Первые шаги, самый простой язык
<code>lesson-01</code> — конспект занятия	После первых экспериментов: что и зачем показывали
<code>transcript</code> — расшифровка записи	Если пропустили занятие или хотите найти конкретный момент
<code>reference</code> — справочник, 50 страниц	Когда упрётесь в конкретный вопрос: настройки, лимиты, безопасность
<code>lesson-01-subtitled.mp4</code>	Сама запись с субтитрами

Две мысли, которые стоит унести до всякой техники:

- Инструмент можно выбрать под себя.** Приложение, расширение в редакторе, браузер, терминал — результат зависит не от этого. Настройки живут в файлах проекта, поэтому пересечь с одного на другое можно в любой момент.
- Настройку можно поручить самому агенту.** Пока вы не разбираетесь, не нужно писать конфиги с нуля: попросите предложить, прочитайте и утвердите. Это рабочая стратегия первых недель — с одной оговоркой, о которой в четвёртом разделе.

Данные о версиях, ценах и лимитах проверялись **12 августа 2026 года**. Это область, где всё меняется за недели: перед важным решением сверьтесь с документацией.

Содержание

- Зачем вам это — что меняется в работе и что получится за первый день
- Где работать: терминал не обязателен — приложение, IDE, браузер, CLI — что выбрать
- Первый запуск — полчаса от нуля до работающего агента
- Пусть агент настроит себя сам — готовые промпты вместо конфигов руками
- Контекст и сессии — почему длинная переписка вредит и что с этим делать
- Чтобы ничего не сломать — разрешения, запреты и проверка результата
- Что дальше — скиллы, MCP, хуки, частые вопросы и первые задачи

1. Зачем вам это

что меняется в работе и что получится за первый день

Честная мысль в начале

Написание кода подешевело. Подорожала **проверка**: основное время разработчика и тестировщика уходит теперь не на набор текста, а на то, чтобы понять, что именно должно получиться, и убедиться, что получилось именно это.

Отсюда вывод, вокруг которого построено занятие: работа с агентом — это не подбор красивых формулировок в запросе, а настройка окружения, в котором агент работает, и организация проверки его результата. Один и тот же запрос в пустой папке и в подготовленном репозитории даёт разный результат, и разница не в словах запроса, а в том, что лежит рядом с кодом.

Чем агент отличается от чата в браузере

Агент — программа, которая работает с файлами вашего проекта напрямую: читает их, правит, запускает команды и показывает, что изменила.

Бытовое сравнение. Чат в браузере — консультант на телефоне: вы диктуете ему кусок кода, он диктует ответ, вы вставляете правку руками, и остальные сорок файлов проекта он не видит. Агент — сотрудник, которого вы посадили за этот проект: он открывает файлы сам, видит соседние, запускает тесты и приносит готовую правку.

Из этого следуют две вещи:

- Агент запускается **в папке проекта** и считает проектом именно её. Соседняя папка для него — чужая территория. Поэтому один и тот же инструмент в двух проектах ведёт себя по-разному: он в них по-разному настроен.
- **Терминал не обязателен.** Claude Code существует как консольная команда, как десктопное приложение для Mac, Windows и Linux (в бете), как веб-версия на `claude.ai/code` и как расширение для VS Code и JetBrains; у Codex тоже есть приложение, консольная версия и расширение. Начинать с той оболочки, где вам спокойнее. Настройки проекта хранятся в файлах внутри самого проекта, а не внутри интерфейса, поэтому переход с одной оболочки на другую ничего не ломает.

Что реально получится после первого дня

Обещаний десятикратного ускорения не будет. Будет вот это:

1. Папка проекта, в которой лежит `CLAUDE.md` — текстовый файл с правилами проекта (стек, соглашения, запреты). Агент читает его автоматически, напоминать о нём в запросе не нужно.
2. Осознанно выбранный режим разрешений: вы понимаете, что агент делает молча, а что — только после вашего подтверждения.
3. Понятная процедура проверки: git-репозиторий, из которого откатывается любая правка, и привычка отдавать результат на ревью второй, независимой модели.

Насколько это окупается, на занятии измерили: независимая проверка **спецификации** — описания задачи, когда кода ещё нет — находит больше и стоит дешевле, чем проверка готового приложения; цифры в разделе «Чтобы ничего не сломать, и как проверять результат».

И сразу приём, который на старте экономит больше всего сил: **пока вы не разобрались, не сочиняйте правила и конфиги руками — попросите агента**. Готовые промпты для этого и разбор того, что

проверять в предложенном, собраны в разделе «Пусть агент настроит себя сам»: ваша роль на первом этапе — согласиться или поправить, а не писать с чистого листа.

Что отдать агенту уже сегодня, а что держать под своим глазом

ОТДАЁМ АГЕНТУ	СМОТРИМ САМИ
Git: ветки, коммиты, слияния, разбор истории, pull requests	Дизайн — самая трудная часть, отсматривается только глазами
Рутинная окрестность: инициализация репозитория, <code>.gitignore</code> , DevOps-команды	Архитектура и выбор стека: это ваши требования, а не его догадки
Тесты на уже написанный код	Всё необратимое: удаление, миграции, деплой, доступ к боевым данным
Разбор незнакомого кода: «объясни, что делает этот модуль»	Итог работы: отчёт агента не заменяет запуск и проверку

Даже отличный результат стоит перепроверить. На занятии сгенерированный `.gitignore` был хорош — и всё равно его прочитали глазами, это заняло несколько секунд.

Как устроен этот комплект материалов

ДОКУМЕНТ	КОГДА ОТКРЫВАТЬ
Стартовый гайд (этот)	Сейчас. Даёт первый рабочий результат за один вечер
<code>lesson-01.md</code> — конспект занятия	После стартового гайда. Тот же путь, но полностью и с деталями демонстрации
<code>reference.md</code> — справочник	По необходимости, как энциклопедия: команды, конфиги, ссылки. Попробуй его читать не нужно

Что делать прямо сейчас

1. Выберите оболочку, в которой вам комфортно — приложение, расширение к редактору или консоль — и установите её (следующий раздел показывает, как).
2. Возьмите свой небольшой **не боевой** проект, откройте его агентом и начните с чтения, а не с правок:

Разберись, как устроен этот проект, и объясни мне его структуру: что где лежит и чем запускается.
Ничего не меняй и не создавай файлы — только прочитай и расскажи.

1. Когда объяснение сойдётся с реальностью, попросите предложить `CLAUDE.md` — готовые промпты для этого в разделе «Пусть агент настроит себя сам».

2. Где работать: терминал не обязателен

приложение, IDE, браузер, CLI — что выбрать

На занятии всё показывали в терминале, и от этого остаётся впечатление, что иначе нельзя. Это не так. Claude Code — один и тот же движок, у которого несколько «окон»: десктопное приложение, панель

внутри вашей IDE, вкладка в браузере, терминал. Задача решается одинаково, отличается только то, что вы видите на экране.

Формы Claude Code

Десктопное приложение (macOS, Windows, Linux в бете) — работа с кодом на кнопках: боковая панель сессий, окно сравнения изменений (diff — подсветка того, какие строки добавлены и удалены) с кнопками «Принять» и «Отклонить», встроенный терминал и просмотр запущенного приложения. Ни Node.js, ни CLI отдельно ставить не нужно, приложение уже содержит Claude Code. Загрузка — claude.com/download.

Расширение для VS Code — панель чата прямо в редакторе, правки показываются в файле. Ставится из магазина расширений по запросу «Claude Code», работает в VS Code 1.94 и выше, а также в Cursor и других сборках на его основе. Внутри у расширения своя копия CLI, поэтому отдельная установка для панели не требуется.

Веб-версия на claude.ai/code — ставить не нужно ничего. Сессия (один разговор с агентом про вашу задачу) идёт на серверах Anthropic и продолжается после того, как вы закрыли вкладку. Код берётся с GitHub и туда же уходит результат: при первом входе понадобится войти через GitHub и разрешить доступ к репозиториям — ко всем сразу или к выбранным вручную. Статус — исследовательский предпросмотр для планов Pro, Max и Team, а также для Enterprise с premium-местами или местами Chat + Claude Code.

Плагин для JetBrains (IntelliJ IDEA, PyCharm, WebStorm, PhpStorm, GoLand, Android Studio) — вызов по `Cmd+Esc` на Mac или `Ctrl+Esc` на Windows и Linux, diff в родном окне сравнения, выделенный фрагмент кода и ошибки инспекции уходят агенту сами. Устройство другое: плагин запускает `claude` во встроенном терминале, поэтому CLI нужно поставить дополнительно.

CLI — программа `claude`, которая запускается в терминале. Тот же агент без графики; нужна для скриптов, удалённых серверов и для плагина JetBrains. Команды установки — в конце раздела.

Есть ещё мобильное приложение Claude для iOS и Android — им удобно запускать и смотреть облачные сессии, но писать код с телефона неудобно.

Документация по всем формам — code.claude.com/docs.

Приложение Claude и Claude Code — не одно и то же

Путаница возникает из-за названий. Скачивается одно приложение — **Claude**, и в нём три вкладки:

- **Chat** — обычный разговор, к вашим файлам доступа нет. Это то, что люди обычно и называют «Клод».
- **Cowork** — самостоятельный агент, работающий в изолированной виртуальной машине.
- **Code** — и есть Claude Code: доступ к вашим локальным файлам, каждое изменение вы подтверждаете.

То есть «десктопный Claude Code» — это вкладка Code внутри приложения Claude. Для неё нужна платная подписка (Pro, Max, Team или Enterprise); на Windows для локальных сессий дополнительно требуется установленный Git.

Сравнение

ФОРМА	КОМУ ПОДОЙДЁТ	ЧТО УСТАНОВИТЬ	ГЛАВНОЕ ОГРАНИЧЕНИЕ
Десктоп-приложение	Тем, кто не хочет терминал вообще	Приложение Claude, вкладка Code	Нет запуска из скриптов и автоматизации
VS Code	Тем, кто не хочет выходить из редактора	Расширение из магазина	Для команд CLI нужна ещё и отдельная установка <code>claude</code>
Веб <code>claude.ai/code</code>	Долгим задачам без постоянного присмотра	Ничего	Код должен лежать на GitHub; с GitLab и Bitbucket не работает
JetBrains	Пользователям IntelliJ, PyCharm и родственных IDE	CLI и плагин с Marketplace	Две установки вместо одной
CLI	Тем, кто живёт в терминале; скрипты, CI, удалённые серверы	<code>claude</code> одной командой	Только текстовый интерфейс

Codex от OpenAI

Устроен так же. Есть приложение для macOS и Windows (Windows — с 4 марта 2026), рассчитанное на несколько агентов параллельно; есть CLI (`curl -fsSL https://chatgpt.com/codex/install.sh | sh`, `brew install --cask codex`); есть расширение для VS Code, Cursor и Windsurf; есть веб на `chatgpt.com/codex`. Codex входит во все планы ChatGPT: на Free и Go — с урезанными лимитами, на Plus, Pro, Business, Enterprise и Edu — в полном объёме. Одним логином открываются все формы сразу. Документация переехала на `learn.chatgpt.com/docs` — старый адрес `developers.openai.com/codex` отдаёт редирект.

Те же агенты живут и в других средах: Cursor (отдельная IDE на базе VS Code), VS Code с GitHub Copilot, Gemini CLI от Google. Подробности — тема отдельного разговора.

Что выбрать

- **«Боясь терминала»** — десктопное приложение, вкладка Code. По умолчанию оно спрашивает разрешение на каждую правку и показывает diff до того, как файл изменится. Если не хочется ничего устанавливать — `claude.ai/code` в браузере.
- **«Живу в IDE»** — расширение для VS Code (одна установка) или плагин для JetBrains (плагин плюс CLI).
- **«Хочу как на занятии»** — CLI, команды установки в конце раздела.

Интерфейс можно поменять в любой момент

Выбор окна не влияет на качество результата. Поведение агента настраивается файлами, а не интерфейсом. Часть их лежит в самом репозитории: файл правил проекта, разрешения в `.claude/settings.json`, каталог `.claude/rules/`. Часть — в вашем домашнем каталоге: `~/.claude/settings.json`, конфигурация MCP-серверов (мостов к внешним инструментам вроде браузера) и установленные скиллы (готовые инструкции под отдельные виды работ) с плагинами. И те и другие читаются всеми локальными формами одинаково, поэтому CLI и приложение можно держать открытыми одновременно на одном проекте. Начали в приложении, потом захотели терминал — переносить нечего.

Эти файлы, кстати, не обязательно писать самому. Попросите агента:

```
Посмотри проект и предложи, что стоит записать в файл правил CLAUDE.md.  
Пока ничего не создавай — сначала покажи черновик и объясни каждый пункт.
```

Предложенное обязательно читайте глазами — что именно там вычитывать, разобрано в разделе «Пусть агент настроит себя сам».

Если всё-таки терминал

CLI ставится одной строкой — на macOS, Linux и в WSL (подсистеме Linux внутри Windows):

```
curl -fsSL https://claude.ai/install.sh | bash
```

В Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

На macOS работает и `brew install --cask claude-code`.

3. Первый запуск

полчаса от нуля до работающего агента

Шаг 0. Что нужно до старта

Платная подписка. Claude Code не входит в бесплатный тариф claude.ai — нужен Pro, Max, Team или Enterprise. Что выбрать и сколько это стоит — в разделе «Что дальше», подраздел «Частые вопросы».

Машина. macOS 13+, Windows 10 (сборка 1809)+ или Linux (Ubuntu 20.04+, Debian 10+), 4 ГБ памяти.

Учебная папка. Не начинайте на рабочем репозитории с ценным кодом. Возьмите копию проекта в отдельном каталоге или пустую папку.

Git под ней — главная страховка: каждая правка агента видна в панели изменений и откатывается назад. Заводится репозиторий и без терминала: в VS Code вкладка **Source Control** → **Initialize Repository**, отдельной программой — GitHub Desktop. В терминале это `git init`. Коммиты тоже не обязательно делать самому: «зафиксируй текущее состояние в git» — нормальная просьба к агенту.

Шаг 1. Установка: три дорожки на выбор

Терминал не обязателен. Claude Code — это один и тот же движок в четырёх обличьях: CLI, десктопное приложение, расширение для редактора и веб (claude.ai/code). Переход с одной дорожки на другую ничего не ломает — начните с той, что ближе.

Дорожка А — десктопное приложение (терминал не нужен)

1. Скачайте установщик со страницы claude.com/download — есть сборки для macOS (Intel и Apple Silicon), Windows (x64 и ARM64), Linux (beta).
2. Запустите приложение и войдите в аккаунт.
3. Сверху три вкладки: **Chat**, **Cowork**, **Code**. Работа с кодом — во вкладке **Code**.

Node.js и CLI отдельно ставить не нужно: приложение несёт Claude Code внутри. На Windows перед первым открытием вкладки **Code** обязателен Git for Windows: поставьте его и перезапустите приложение, иначе локальные сессии не запустятся.

Дорожка Б — расширение для редактора

VS Code и его форки (Cursor, Windsurf — редакторы, собранные на его основе): `Cmd+Shift+X` (Mac) или `Ctrl+Shift+X` (Windows/Linux) → поиск «Claude Code» → Install. Затем палитра команд `Cmd+Shift+P` / `Ctrl+Shift+P` → «Claude Code» → **Open in New Tab**.

JetBrains (IntelliJ IDEA, PyCharm, WebStorm, PhpStorm, GoLand, Android Studio): плагин Claude Code из Marketplace, потом перезапуск IDE. Плагину нужен установленный CLI — поставьте его по дорожке В.

Дорожка В — CLI в терминале

macOS, Linux, WSL (подсистема Linux внутри Windows):

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Через Homebrew на macOS: `brew install --cask claude-code`.

Проверка, что всё встало:

```
claude --version    # печатает номер, например: 2.1.228 (Claude Code)
claude doctor       # диагностика установки и настроек, сессию не запускает
```

Установка через инсталлятор обновляется в фоне сама. Homebrew и WinGet — нет, там обновление руками. Git for Windows для CLI не обязателен: без него агент выполняет команды через PowerShell, а не через Bash.

У Codex от OpenAI набор тот же: приложение, расширение для VS Code и его форков (Cursor, Windsurf) и CLI (`curl -fsSL https://chatgpt.com/codex/install.sh | sh`).

Шаг 2. Открыть папку проекта

Агент работает внутри одной папки — она для него и есть «проект». Он видит файлы в ней и правила, которые в ней лежат; соседние проекты в его картину мира не попадают. Поэтому один и тот же Claude Code в разных папках ведёт себя по-разному.

ГДЕ ВЫ	КАК ВЫБРАТЬ ПАПКУ
CLI	<code>cd ~/projects/uchebnyj && claude</code>
Приложение	вкладка Code → окружение Local → кнопка Select folder
Расширение	берётся папка, открытая в редакторе

При первом запуске откроется браузер для входа в аккаунт.

Шаг 3. Первая задача — маленькая и безопасная

Начните с задачи, где агент только читает:

Осмотри в этой папке: какие тут файлы, на чём написан проект, как его запустить. Ничего не меняй — расскажи, что увидел, и скажи, каких сведений тебе не хватает.

Если ответ по делу — связка работает. Теперь маленькая правка:

Добавь в README.md короткий раздел «Как запустить» на основе того, что нашёл. Сначала покажи, что именно собираешься записать.

И третья — специально отказная. Попросите то, на что стоит ответить «нет»:

Удали README.md, он мне больше не нужен.

Агент назовёт файл и дожждётся ответа. Откажите. Ничего не сломается: отказ приходит к нему как обратная связь, и дальше он предложит другой путь — перенести содержимое или переименовать файл.

Шаг 4. Что вы увидите на экране

Агент проговаривает каждый свой шаг, и это стоит читать, а не пролистывать:

- **Что он читает** — строки вида `Read(src/app.js)`. Так видно, на чём он строит выводы.
- **Какую команду хочет выполнить** — команда показывается целиком, и он ждёт вашего ответа: разрешить один раз, разрешить всегда, отказать.
- **Что меняет в файлах** — diff: было слева, стало справа. Посмотрите на селектор режима рядом с полем ввода. Если там **Manual** — файл не изменится, пока вы не нажмёте **Accept**. Если **Auto** — агент правит сам; на первые дни верните **Manual** тем же селектором (`Cmd+Shift+M` открывает список режимов). Про режимы целиком — раздел «Чтобы ничего не сломать».

Как отменить. Три ситуации — три разных действия:

- **Правка ещё не принята** — кнопка **Reject** на окне сравнения. Файл остаётся как был.
- **Принята, но не закоммичена** — панель изменений: в **Source Control** редактора это **Discard all changes**, в GitHub Desktop — **Discard changes** в списке изменённых файлов. В терминале то же самое делает `git restore`.
- **Агент ещё работает** — прервать: `Esc` в CLI, кнопка стоп в приложении.

Откат возвращает файлы к последнему коммиту — поэтому он и работает только тогда, когда коммит перед началом работы был сделан.

Ещё одна страховка на первые дни: `Shift+Tab` в CLI переключает режимы разрешений, среди них **plan** — в нём агент только описывает план и не трогает файлы.

Шаг 5. Если что-то пошло не так

СИМПТОМ

ЧТО ДЕЛАТЬ

<code>command not found: claude</code>	Закрыть терминал и открыть заново — путь к программе подхватывается при старте. Не помогло — страница code.claude.com/docs/en/troubleshoot-install
Не пускает в аккаунт, ошибка 403	Проверить, что вошли под аккаунтом с платной подпиской; в приложении — выйти, войти заново и перезапустить его
Ведёт себя странно после установки	Наберите <code>/doctor</code> в поле ввода — список команд открывается по <code>/</code> . Вариант для CLI: <code>claude doctor</code> . Покажет состояние установки и ошибки в настройках, ничего при этом не меняя
Отказали в доступе, и работа встала	Напишите словами, что можно вместо этого: «не запускай ничего, покажи текст изменений»
Не видит нужный файл	Проверьте, из той ли папки запущена сессия; отдельный файл подтягивается упоминанием <code>@путь/к/файлу</code>

Отдельный приём, который экономит вечер: не пишите настройки руками, пока не разобрались. Попросите:

Посмотри этот проект и предложи, что стоит записать в CLAUDE.md — файл с правилами для тебя. Сначала объясни, что собираешься написать и почему, и дожись моего согласия.

Оговорка обязательная: предложенное читайте глазами — что именно вычитывать, разобрано в разделе «Пусть агент настроит себя сам».

Готово, если...

- ☐ в приложении открыта вкладка **Code** и выполнен вход в аккаунт (в терминале то же подтверждает `claude --version`);
- ☐ сессия запущена из учебной папки, и агент верно описал, что в ней лежит;
- ☐ вы видели диалог запроса разрешения и хотя бы раз ответили «нет»;
- ☐ одна маленькая правка принята и видна в панели изменений приложения или во вкладке **Source Control** редактора;
- ☐ вы знаете, как прервать агента, как откатить принятую правку и где смотреть диагностику.

4. Пусть агент настроит себя сам

готовые промпты вместо конфигов руками

Настройка проекта под агента выглядит как отдельная дисциплина: файлы правил, разрешения, скиллы. Но это такая же задача, как «почини баг», — и агент умеет её выполнять. Пока вы не разобрались в деталях, писать конфиги руками не нужно: попросите агента осмотреть проект и предложить, а сами выступите заказчиком и приёмщиком.

Вы читаете предложенное, вычёркиваете лишнее, дописываете своё — минуты вместо вечера, и попутно становится понятно, из чего настройка состоит. Всё это лежит в файлах внутри проекта, а не в интерфейсе: настроили из десктопного приложения — те же файлы работают в расширении для VS Code, в вебе и в терминале.

Что такое файл правил проекта

CLAUDE.md — обычный markdown-файл в корне проекта: памятка для агента, которую он читает сам, в начале каждой сессии, без напоминания в промпте. Стек и версии, команды сборки и тестов, соглашения, запреты. Смысл — не повторять одно и то же в каждом сообщении.

Второй формат — **AGENTS.md**, открытая конвенция, которую понимают Codex, Cursor, Copilot, Gemini CLI и ещё десятка два инструментов. Claude Code читает только **CLAUDE.md**, но связать их можно одной строкой **@AGENTS.md** внутри **CLAUDE.md**. Рекомендуемый размер — до 200 строк; крупное выносят в отдельные файлы и ссылаются на них.

Агент сам такой файл не создаёт, и пока файла нет, он в свободном полёте. Быстрый старт — команда **/init**: она осмотрит код и запишет файл сразу, черновик заранее не показывая. Хотите увидеть и поправить каждый пункт до записи — идите промптами ниже.

Команды со слэшем набираются в том же поле, где вы пишете агенту, отдельный терминал для них не нужен. Доступны они не везде одинаково: в CLI и десктопном приложении есть все, в панели VS Code — подмножество (наберите слэш, чтобы увидеть список), в веб-версии части команд нет.

Готовые промпты

1. Осмотреть проект и предложить файл правил.

Осмотри проект: структуру каталогов, файлы сборки, зависимости, тесты и скрипты запуска. Пока ничего не меняй. Затем предложи черновик **CLAUDE.md**: стек и версии как они есть в коде, команды сборки, тестов и линтера, соглашения по именованию и структуре, чего в этом проекте делать нельзя. Пиши только то, что подтверждается файлами. Всё, о чём приходится догадываться, вынеси в конец отдельным списком «Требуется подтверждения».

Получите черновик и список догадок. Проверьте, что все библиотеки и команды действительно есть в проекте.

Шаг приёмки

Черновик в переписке файлом не становится: файл появится, когда вы попросите. Ответьте обычной репликой — что оставить, что убрать, что записать.

Всё верно, кроме пункта 4 — убери его. Создай файл правил в корне проекта с остальным и покажи, что записал.

Дальше агент попросит разрешение на запись: в приложении это кнопка принятия над сравнением изменений, в CLI — вопрос с вариантами ответа. Согласились — убедитесь, что получилось: файл лежит в корне проекта, откройте его и перечитайте целиком.

2. Объяснить каждый пункт простыми словами.

Пройди по предложенному файлу по пунктам. Для каждого скажи одной фразой: что он заставляет тебя делать, откуда ты его взял (файл и строка или «моё предположение») и что изменится в работе, если пункт убрать. Ничего не исправляй, просто объясни.

Пункты со словом «предположение» — кандидаты на удаление.

3. Проверить уже существующий файл.

Прочитай CLAUDE.md в корне и разбери его критически: какие правила противоречат друг другу, какие устарели относительно кода, какие сформулированы так мягко, что их можно обойти, и каких важных правил не хватает. Покажи правки списком «было — стало» и жди моего решения по каждой. Сам файл не редактируй.

4. Настроить разрешения.

Предложи для этого проекта содержимое .claude/settings.json: что запретить полностью, что спрашивать у меня каждый раз, что разрешить без вопросов. Для каждой строки напиши обычным языком, что она разрешает или запрещает и какой случай ею закрывается. Начни с самого строгого варианта.

Проверьте, что в запретах есть чтение .env и ключей, `rm -rf`, `git reset --hard` и force push. Правила читаются в порядке «запрет → спросить → разрешить», первое совпадение выигрывает. Список согласован — попросите записать его в .claude/settings.json и подтвердите запись.

5. Подобрать скилл под стек. Скилл — папка с инструкцией, которая включается, когда задача ей подходит: готовая компетенция агента. Плагин — упаковка, в которой скиллы распространяются и ставятся.

Посмотри стек этого проекта и скажи, какие скиллы ты сейчас видишь и какой из них здесь пригодился бы. По каждому — одной фразой: что он добавляет к твоей работе и на каких задачах включается. Ничего не устанавливай.

Чего под рукой не оказалось — ищите в официальном каталоге: команда `/plugin` в поле ввода открывает список наборов и показывает состав каждого до установки.

6. Объяснить намерение до действия.

Прежде чем что-то менять, опиши: какие файлы создашь, какие изменишь, какие удалишь и почему. Отдельно назови решения, которые принимаешь за меня, потому что я их не оговорил. Дождись моего «да».

7. Оформить повторяющуюся процедуру в скилл.

Мы третий раз повторяем одну и ту же процедуру: <опишите своими словами>. Оформи её скиллом: короткий SKILL.md с описанием, когда его применять, и пошаговой инструкцией. Не пересказывай то, что ты и так знаешь, — только специфику нашего проекта. Сначала покажи текст, установим потом.

Текст устроил — попросите сохранить его скиллом и показать, куда легли файлы.

8. Проверить свою же работу.

Проверь сделанное и покажи саму проверку, а не вывод: какие команды запустил, что они вывели, какие файлы перечитал после правки. Отдельно перечисли, что осталось непроверенным и почему.

Сгенерированный файл читаем глазами — обязательно

Агент дописывает в правила то, чего вы не просили: архитектуру, стек, критерии приёма. Потом он этим правилам аккуратно следует, и вы получаете результат, которого не заказывали. На занятии из-за одной незамеченной строки в файле агент наплодил кучу файлов там, где просили один `index.html`.

Правило общее для всего, что агент записал по промптам выше: файла правил, настроек, текста скилла. В файле правил вычитывают пункт за пунктом:

ПРОВЕРКА	ПОЧЕМУ
Стек и версии	Модель вписывает свой любимый, а не ваш
Требования, которых вы не ставили	«Каждый компонент отдельным файлом», «покрытие 90%» — потом это исполняется буквально
Команды	Скопируйте и запустите: несуществующая команда в файле хуже её отсутствия
Формулировки запретов	«Старайтесь не удалять» игнорируется; «ни при каких обстоятельствах не удалять» — работает
Остатки прошлых задач	Каждая живая строка влияет на результат

И честная граница: запрет словами — просьба, а не гарантия. Настоящий запрет живёт в разрешениях (`deny`), а не в тексте правил. Ещё одно: правки файла подхватываются со следующей сессии, менять его посреди работы агента не стоит.

Сначала план, потом правки

Самая дешёвая проверка из всех. Переключите режим планирования (`Shift+Tab` циклит режимы) или попросите словами: «сначала план, код после моего согласия». В плане сразу видно то, что в готовом коде искать долго: лишние файлы, чужие библиотеки, задачи, которых вы не ставили.

Как расти дальше

1. **Агент предлагает — вы утверждаете.** Первые недели: все промпты выше, ничего руками.
2. **Вы правите — агент объясняет последствия.** Начинаете дописывать свои правила и спрашивать, как они изменят поведение.
3. **Вы пишете — агент проверяет.** Файл правил становится вашим, а промпт №3 превращается в регулярный аудит: раз в месяц или после крупного изменения в проекте.

5. Контекст и сессии

почему длинная переписка вредит и что с этим делать

Контекст — это рабочий стол, а не архив

Контекст (или «контекстное окно») — всё, что агент держит перед глазами сейчас. Не хранилище, куда знания складываются навсегда, а стол ограниченного размера: что на нём лежит — то он видит; что не поместилось — того для него нет.

На этот стол попадает далеко не только ваше сообщение:

- системная инструкция агента и описания всех его инструментов;
- правила проекта — файл `CLAUDE.md` (у Codex — `AGENTS.md`);
- описания подключённых расширений: скиллов и MCP-серверов — программ, которые дают агенту доступ к внешним инструментам;
- каждый прочитанный агентом файл;
- вывод каждой запущенной команды, включая длинные логи и трейсы;
- вся история переписки с начала сессии — ваши реплики, его ответы, написанный код.

Ваш промпт — небольшая часть этого объёма. Качество ответа определяется не столько формулировкой запроса, сколько тем, что вообще лежит на столе. У Claude окно — миллион токенов. Токен — кусочек текста размером с часть слова, так что на стол помещаются сотни тысяч слов, несколько толстых книг. Запас кажется бездонным, но код, длинные логи и история переписки расходуют его быстрее, чем ожидаешь.

Почему длинная переписка мешает

Интуиция подсказывает: чем больше агент знает о задаче, тем лучше. На практике наоборот. Внимание модели распределяется по всему, что лежит на столе, и чем там больше всего, тем тоньше слой достаётся нужному. Замеры на десятках моделей дают один результат: с ростом объёма точность падает, причём хуже всего читается середина — начало и конец удерживаются лучше. Одного лишнего отвлекающего фрагмента бывает достаточно, чтобы ответ стал хуже.

Практический вывод: три часа переписки, в которой вы чинили другой баг, — не помощь новой задаче, а помеха. Старые версии файлов, отменённые решения и ошибочные гипотезы выглядят для агента такими же актуальными, как ваша последняя фраза.

Что происходит при переполнении

Когда место заканчивается, срабатывает компактификация (в интерфейсе — «компакт»): агент заменяет историю её сжатым пересказом и продолжает работать с ним. Пересказ он пишет сам, и вместе с водой уходит часть подробностей — почему выбрали этот подход, какой вариант уже отбросили, как точно называлось поле в базе.

Отсюда знакомое ощущение, что после долгой сессии агент «стал тупить». Он не изменился — у него забрали половину заметок. Особенно неприятно, если компакт прошёл, пока вы отходили от монитора: вы продолжаете разговор в уверенности, что он всё помнит, а он опирается на конспект.

Три правила гигиены

1. Новая задача — новая сессия. Закончили одно — очистите контекст командой `/clear` и начните с чистого листа. Это не стоит ничего и снимает риск, что агент потянет в новую задачу мусор из прошлой. В CLI и десктопном приложении доступны все команды; в панели VS Code — часть, наберите слэш и посмотрите список; в веб-версии `/clear` нет — там новую сессию заводят в самом интерфейсе.

2. Большую задачу дробите. «Сделать авторизацию» — это несколько сессий: схема данных, эндпоинты, экран входа, тесты. Каждый кусок помещается в свежий контекст целиком, и результат вы проверяете, пока он ещё обозримый.

3. Не чините одно и то же по кругу. Если две-три попытки исправить баг не сработали, все неудачные варианты уже лежат в контексте, и агент будет ходить вокруг них. Круг разрывается перезапуском с нормальным описанием проблемы — а составить это описание может он сам:

Мы третий раз не можем починить эту ошибку. Не предлагай новых правок.
Напиши сжато для новой сессии: в чём симптом, что уже пробовали
и почему это не сработало, какие файлы затронуты.

Ответ скопируйте, дайте `/clear` и начните заново с этого текста.

Как посмотреть, чем занят контекст

Команда `/context` показывает заполнение окна цветной сеткой: сколько занимает системная часть, сколько — правила проекта, описания инструментов и скиллов, сколько — сама переписка и близко ли предел. Она же подсказывает, какой инструмент расходует больше всего; частая находка — забытый MCP-сервер, не нужный в этом проекте.

Терминал для этого открывать не обязательно: в десктопном приложении `/context` работает так же.

Что делать, когда предупреждают о переполнении

Лучше не дожидаться автоматике, а вызвать `/compact` самому в естественной паузе — когда кусок работы закончен. Главное: компакту можно указать, что сохранить, — допишите инструкцию текстом:

`/compact` сохрани формулировку задачи, список изменённых файлов
и то, какие подходы мы уже отвергли и почему

Если задача завершена, предпочтительнее `/clear`: полная очистка честнее пересказа.

Субагент: вынести возню из переписки

Иногда нужно перелопатить много всего: найти по репозиторию все вызовы функции, разобрать стостраничный лог, сверить документацию. В основной сессии весь этот мусор осядет на столе.

Субагент — отдельный экземпляр агента со своим контекстом, которому такую работу поручают. Он читает и ищет у себя, а в вашу переписку возвращает только итоговую сводку. Просить можно обычными словами:

Поручи субагенту найти все места, где вызывается `processPayment`,
и вернуть только список файлов со строками. Не читай их в основной сессии.

Симптом → что происходит → что сделать

СИМПТОМ	ЧТО ПРОИСХОДИТ	ЧТО СДЕЛАТЬ
Ответы стали общими, ранние договорённости забыты	Прошёл компакт, подробности заменены пересказом	<code>/clear</code> и заново с коротким описанием задачи
Агент правит один файл по кругу, ломая уже исправленное	В контексте лежат все неудачные попытки	Попросить сводку по проблеме, затем <code>/clear</code> и старт с ней
Предупреждение о близком переполнении	Место на столе кончается	<code>/compact</code> с указанием, что сохранить, или <code>/clear</code> , если задача закрыта
Агент ссылается на решение из прошлой задачи	Сессия не была начата заново	<code>/clear</code> перед каждой новой задачей
Окно заполнено ещё до первого сообщения	Подключено много MCP-серверов, разросся <code>CLAUDE.md</code>	<code>/context</code> , отключить лишние серверы, ужать правила
После команды с простыней логов всё пошло хуже	Вывод целиком лёг в контекст	Такие разборы поручать субагенту

6. Чтобы ничего не сломать

разрешения, запреты и проверка результата

Агент спрашивает разрешения не всегда: сколько именно — настройка, которую вы задаёте сами. Дальше — как её выставить и как потом убедиться, что сделанное действительно сделано.

Режимы разрешений

Режим разрешений — базовая линия: что агент выполняет без вопроса, а что выносит на подтверждение.

РЕЖИМ	ЧТО ДЕЛАЕТ БЕЗ ВОПРОСА	КОМУ И КОГДА
Ручной (<code>default</code> , в интерфейсе — Manual)	только читает файлы	первые дни, чужой или чувствительный код
Планирование (<code>plan</code>)	изучает проект и предлагает план; правки заблокированы, пока вы план не утвердите	разбор задачи перед началом работы
Одобрение правок (<code>acceptEdits</code>)	правит файлы в рабочей папке; остальное спрашивает	понятная задача внутри git-репозитория
Auto	почти всё, но каждое действие проверяет встроенный классификатор	длинные задачи, когда доверие уже набрано
Bypass permissions	не спрашивает ничего	только внутри контейнера или виртуальной машины

Как переключать. `Shift+Tab` циклит между ручным, одобрением правок и планированием; текущий режим виден внизу экрана. В панели VS Code параметр `defaultMode` из файла настроек не действует — режим там выбирается индикатором рядом с полем ввода. В веб-версии ручного режима нет вовсе.

Про bypass. В CLI перейти в него из сессии, стартовавшей без него, нельзя: он задаётся флагом при запуске. В десктопном приложении это, наоборот, обычный переключатель, доступный посреди работы, — защита там слабее, и трогать его не нужно.

Запреты и разрешения из `.claude/settings.json` действуют одинаково во всех формах. Стартовый режим — исключение: его читают не все интерфейсы.

Рекомендация на первый месяц: планирование для разбора задачи, затем одобрение правок — и частые коммиты. Так вы видите план до работы и `git diff` после неё. С 14 августа 2026 auto становится режимом по умолчанию для новых сессий на Pro, Max и Team, так что свой режим стоит прописать явно.

Случай с занятия: почему запрет словами не сработал

В файле правил проекта было написано «не удалять папку Temp». Агент папку удалил — и в том же ответе сообщил, что удалять её нельзя.

Сессия шла в режиме `bypass permissions`, где не работают ни подтверждения, ни классификатор. А файл правил — часть текста, который читает модель: он влияет на то, что агент *собирается* сделать, но не на то, что ему *позволено*. Проверку выполняет сама программа, и в списке того, что она смотрит, файла правил нет.

Вывод простой: просьба в файле правил — это просьба. Механизм — режим разрешений и список запретов в настройках. Формулировки тоже влияют (после замены на «ни при каких обстоятельствах» агент отказался выполнять команду), но опираться стоит на механизм.

Минимальный набор запретов с первого дня

Нужен файл `.claude/settings.json`. Папка `.claude` скрытая — в Finder её не видно и через «Новую папку» не создать. В VS Code или Cursor: File, New File и ввести путь целиком, `.claude/settings.json`, — папку редактор создаст сам. В приложении — попросить агента: «создай `.claude/settings.json` с таким содержимым», а самому прочитать diff и нажать «Принять».

Содержимое:

```
{
  "permissions": {
    "defaultMode": "plan",
    "disableBypassPermissionsMode": "disable",
    "deny": [
      "Read(**/.env)",
      "Read(**/.env.*)",
      "Read(~/.ssh/**)",
      "Bash(rm -rf *)",
      "Bash(git reset --hard*)",
      "Bash(git push --force*)",
      "Bash(curl *)",
      "Bash(wget *)"
    ],
    "ask": ["Bash(git push *)"]
  }
}
```

СТРОКА	ЗАЧЕМ
<code>Read(**/.env)</code> , <code>Read(~/.ssh/**)</code>	секреты и ключи — самый частый канал утечки; запрет на чтение закрывает и запись
<code>rm -rf</code> , <code>git reset --hard</code> , <code>git push --force</code>	необратимая потеря того, что не закоммичено
<code>curl</code> , <code>wget</code>	выход в сеть в обход остальных ограничений
<code>disableBypassPermissionsMode</code>	вы сами закрываете себе режим «не спрашивать ничего»

Файл читается при старте сессии: поправили — перезапустите её. Что действует сейчас, показывает команда `/permissions`.

Честная оговорка: такие правила — фильтр от небрежности, а не граница безопасности. `rm -rf *` не поймает написание `rm -fr`.

Но безопасность первого месяца держится не на них, а на том, что у вас уже есть: режим, в котором агент спрашивает перед каждым действием; коммит перед началом каждой задачи, к которому всегда можно вернуться; учебная папка без боевых данных. Запреты в настройках — ещё один слой поверх этого, он ловит случайности. Настоящую границу дают песочница и **жуки** — ваш код, который запускается перед действием агента и может его заблокировать: `reference.md`, разделы 5 и 7.

Не отдавайте агенту вообще: продакшн-базу и продовые доступы (в июле 2025 агент Replit удалил боевую базу вопреки объявленному запрету на изменения), необратимые операции за пределами вашей машины (`terraform destroy`, `kubectl delete`), файлы с секретами.

Набор можно подобрать под проект:

Посмотри проект и предложи `.claude/settings.json`: запреты на чтение секретов и на необратимые команды. Объясни каждую строку отдельно. Ничего не применяй, пока я не соглашусь.

Предложенное вычитайте по чек-листу из раздела 4: лишняя строка в разрешениях работает наравне с нужной.

«Готово» ещё не значит готово

Агент склонен объявлять работу законченной по факту написания кода, а не по факту его запуска. Поэтому требуйте доказательство — вывод реально выполненной команды.

Не пиши «готово». Запусти проект и тесты, покажи вывод команд целиком. Если что-то запустить не удалось — скажи прямо, что осталось непроверенным.

Чек-лист на любое изменение:

- 1. То ли сделано?** Сверьте результат с тем, что просили, а не с тем, что агент пересказал.
- 2. Не задето ли лишнее?** `git diff --stat` и `git status` — тот же список показывает панель изменений в редакторе и в приложении: файлы, которых вы не ждали, — повод остановиться.
- 3. Работает ли то, что работало раньше?** Тесты и запуск приложения, а не только новый экран.

Покажи `git diff --stat` и по каждому файлу объясни, зачем он изменён. Отдельно перечисли изменения, о которых я не просил.

Независимая проверка другой моделью

Второй агент смотрит работу первого свежим взглядом: он не участвовал в обсуждении, не привязан к принятым решениям и не защищает свой результат.

Проверка **спецификации**, когда кода ещё нет, заняла на занятии около двух минут и дала **13 находок, все по делу**: от зазоров, посчитанных не от того элемента, до ввода, который не блокировался при экране победы. Чек-лист вырос с 9 до 22 пунктов — до написания единой строки кода. Проверка готового приложения — уже 5–6 минут, и на большом коде разрыв растёт. Отсюда правило: чем раньше вы зовёте вторую модель, тем дешевле обходятся её находки.

Словами, в текущей сессии. Claude Code вызывает Codex сам, терминал открывать не нужно:

```
Проверь файл docs/specs/<имя>.md через Codex: логические ошибки,
противоречия, пробелы и неверные технические утверждения.
Верни находки списком, ничего не исправляй.
```

Условие честное: Codex должен быть установлен, и нужна действующая подписка ChatGPT — вторая платная сверх Claude. Для учёбы не обязательна.

Без второй подписки — чистая сессия того же агента. Дайте `/clear` или откройте новую и не пересказывайте в ней обсуждение: ревьюер должен видеть только файлы.

```
Ты независимый ревьюер, эту работу делал не ты. Прочитай <путь>, сверь
с требованиями в <путь> и перечисли находки по severity: критично /
важно / мелочь. Правок не вноси.
```

Слепые зоны у одной модели общие, зато привязанность к своим решениям пропадает.

Из терминала, если так привычнее:

```
codex exec --skip-git-repo-check "Прочитай файл docs/specs/<имя>.md — это
спецификация. Найди логические ошибки, противоречия, пробелы и неверные
технические утверждения. Ничего не исправляй, только перечисли находки"
```

7. Что дальше

скиллы, MCP, хуки, частые вопросы и первые задачи

Три вещи, о которых стоит знать — но настраивать не сегодня

Скилл — папка с текстовым файлом, где записана одна компетенция агента: как делать определённую работу и когда за неё браться. Агент постоянно видит только короткое описание, а текст читает, когда задача с описанием совпала. На занятии эффект показали вживую: без скилла модель сама выбрала оформление и выдала бледную страницу, а со скиллом `frontend-design` по той же задаче подобрала палитру, шрифты и анимации. Скилл не добавляет знаний, он задаёт роль и планку. `reference.md`, раздел 2.

MCP — способ дать агенту доступ к внешнему инструменту: браузеру, актуальной документации библиотеки, дизайн-канвасу. Без этого он знает то, что было в обучении, и то, что лежит в репозитории. С этим — открывает страницу, нажимает кнопки, читает консоль, сверяется с

документацией нужной версии. Обратная сторона: MCP-сервер — программа, запускаемая с вашими правами, поэтому подключать стоит немного и от понятного издателя. [reference.md](#), раздел 4.

Хук — ваш код, который агент запускает сам в заданный момент: перед каждой командой, после каждой правки файла, в начале сессии. Отличие от правил в тексте принципиальное: текст — просьба, которую модель может не удержать в фокусе, а хук выполняется всегда. Отсюда и технический запрет, обещанный в разделе «Чтобы ничего не сломать»: опасную команду хук останавливает, а не отговаривает. Второе типовое применение — прогнать форматтер после правки. [reference.md](#), раздел 5.

Ничего из трёх не нужно писать руками на старте. Порядок тот же, что в разделе «Пусть агент настроит себя сам»: попросить агента предложить, прочитать глазами, согласиться или поправить.

Посмотри проект и предложи один хук или один скилл, который дал бы здесь больше всего пользы. Сначала объясни словами, что ты собираешься сделать и зачем, потом покажи файл целиком. Я скажу, ставить или нет.

Когда задача больше одного промпта

Для задачи в несколько дней последовательность такая: **обсудили** → **записали спецификацию** → **проверили её** → **составили план** → **написали код** → **проверили результат**. Спецификация здесь — короткий документ на понятном языке: что делаем, что не делаем, как поймём, что готово.

Вкладываться нужно в первые три звена. Вычеркнуть придуманную агентом лишнюю функцию на этапе обсуждения стоит одной фразой, на этапе плана — абзаца. Когда код уже написан, та же правка стоит переписанного кода, а иногда и тестов вокруг него.

Давай продумаем задачу до кода. Задавай мне уточняющие вопросы по одному за сообщение, пока не останется неясностей. Потом предложи 2–3 варианта решения с плюсами и минусами и порекомендуй один.

Запиши согласованное в файл `docs/спец-<название>.md`: цель, что входит в объём, что не входит, критерии готовности. Затем перечитай свой же файл и найди в нём противоречия, заглушки и двусмысленные формулировки.

Отдельный приём — показать готовую спецификацию другому агенту: «проверь файл и скажи, что здесь непонятно исполнителю». Инструмент, который проводит по всей цепочке автоматически, существует и разобран в [reference.md](#), раздел 3.

Частые вопросы

Сколько это стоит и что брать для учёбы. Цены проверены 12.08.2026. Claude: Pro \$20/мес (\$17 при годовой оплате), Max 5x — \$100, Max 20x — \$200. ChatGPT/Codex: Go \$8, Plus \$20, Pro от \$100. Для учёбы хватает одной подписки уровня \$20. Оговорка: Anthropic не публикует точных квот, только множители (Pro 1x, Max 5x и 20x); лимит сбрасывается каждые пять часов, у Max сверху недельный потолок. [reference.md](#), раздел 6.

В компании нельзя отправлять код наружу. Решаете не вы — спросите у безопасности, есть ли одобренный вариант: корпоративные тарифы и варианты развёртывания есть у обоих инструментов. Пока разрешения нет, учитесь на пет-проекте или на открытом коде: навык переносится полностью, а настройки — обычные файлы, часть в репозитории проекта, часть в домашнем каталоге, и переезжают вместе с вами.

Агент выдумывает. Выдумывает, и это не лечится. Лечится проверкой: тесты, запуск, второй агент на ревью. Правило одно — не верить слову «готово» без вывода реально выполненной команды.

Заменит ли это меня. Агент не решает, что именно надо построить, и не отвечает за последствия. Обе работы остаются вашими и дорожают по мере того, как кода производится больше.

Нужно ли уметь терминал. Нет. Формы без терминала — приложение, панель в редакторе, браузер — разобраны в разделе «Где работать». Начинайте с привычного: локальные формы читают одни и те же файлы настроек, поэтому переход обходится недорого.

Результат не нравится, а объяснить не получается. Не объясняйте — покажите: ссылку на пример, скриншот, файл в проекте, сделанный правильно. Второй ход — попросить три варианта и выбрать. Третий — «задай мне пять вопросов, чтобы понять, что меня здесь не устраивает».

Пять задач для первой недели

1. Разобраться в незнакомом коде.

Объясни, что делает файл <путь>: сначала абзац в целом, потом по функциям. Отдельно назови места, где чаще всего ошибаются.

2. Написать тесты на существующую функцию.

Напиши тесты для функции <имя> в <путь>. Используй тестовый фреймворк, который уже есть в проекте. Покрой граничные случаи. Функцию не меняй. Запусти тесты и покажи вывод.

3. Привести в порядок .gitignore.

Проверь .gitignore этого проекта: что отслеживается зря, что должно игнорироваться, но не игнорируется, не попали ли в репозиторий файлы с секретами. Изменения предложи списком, не применяй сразу.

4. Разобрать баг.

Есть баг: <что делаю> → <что ожидаю> → <что происходит>. Найди причину. Сначала покажи, где именно ломается и почему, с ссылками на строки. Фикс не пиши, пока я не подтвержу диагноз.

5. Оформить повторяющуюся процедуру в правила проекта.

Я третий раз объясняю тебе одно и то же: <опишите процедуру своими словами>. Предложи, как записать это в CLAUDE.md, чтобы больше не повторять. Покажи текст целиком — я его прочитаю и поправлю.